

Concurrent



Multi-Tasking like a Pro



Mit [synchronized](#) lassen sich Threads **feingranular für maximale Performance thread-safe** machen.

Häufig ist es eine feingranulare Optimierung nicht nötig, dann ist es einfacher, Utility-Klassen zu verwenden, wie z. B.

[SynchronizedCollection](#), [Atomics](#), [ReentrantReadWriteLock](#), [Parallel-Streams](#).

Weitere nützliche Klassen befinden sich im [Concurrent-Package](#).

Synchronized Collections



Die Utility-Klasse [Collections](#) stellt unter anderen Collection-Klassen zur Verfügung, die **thread-safe** sind:

- *synchronizedCollection(Collection<T> c)*
- *synchronizedList(List<T> l)*
- *synchronizedMap(Map<K,V> m)*
- *synchronizedSet(Set<T> s)*

Diese Methoden geben jeweils eine entsprechende Collection-Klasse zurück, bei der alle Methoden (z. B. add, remove, forEach) **thread-safe** sind.

Synchronized Collections: Beispiel

```
record Book(int id) {
```

```
void main() throws Exception {
```

```
// Alle Methoden in synchronizedList sind intern durch synchronized geschützt.
```

```
List<Book> books = Collections.synchronizedList(new ArrayList<>());
```

```
Thread t1 = new Thread(() -> {
```

```
    for (int i = 0; i <= 1000; i++) {
```

```
        books.add(new Book(i)); // add ist thread-safe, da books eine synchronizedList ist.
```

```
    }
```

```
});
```

```
Thread t2 = new Thread(() -> {
```

```
    for (int i = 1001; i <= 2000; i++) {
```

```
        books.add(new Book(i)); // add ist thread-safe, da books eine synchronizedList ist.
```

```
    }
```

```
});
```

```
t1.start();
```

```
t2.start();
```

```
t1.join();
```

```
t2.join();
```

```
IO.println(books);
```

```
}
```

```
}
```

Mögliche Ausgabe:

```
[Book[id=0], Book[id=1], Book[id=2], Book[id=3], Book[id=4], Book[id=5], Book[id=6], Book[id=7],  
Book[id=8], Book[id=9], Book[id=10], Book[id=11], Book[id=12], Book[id=13], Book[id=14],  
Book[id=15], Book[id=1001], Book[id=16], Book[id=17], Book[id=1002], Book[id=18],  
Book[id=1003], Book[id=19], Book[id=1004], Book[id=20], Book[id=1005], Book[id=21],  
Book[id=1006], Book[id=22], Book[id=1007], Book[id=23], Book[id=1008], Book[id=24],  
Book[id=1009], Book[id=1010], Book[id=1011], Book[id=1012], Book[id=1013], Book[id=1014],  
Book[id=1015], Book[id=1016], Book[id=1017], Book[id=1018], Book[id=1019], ...
```

Future<V>



Quelle

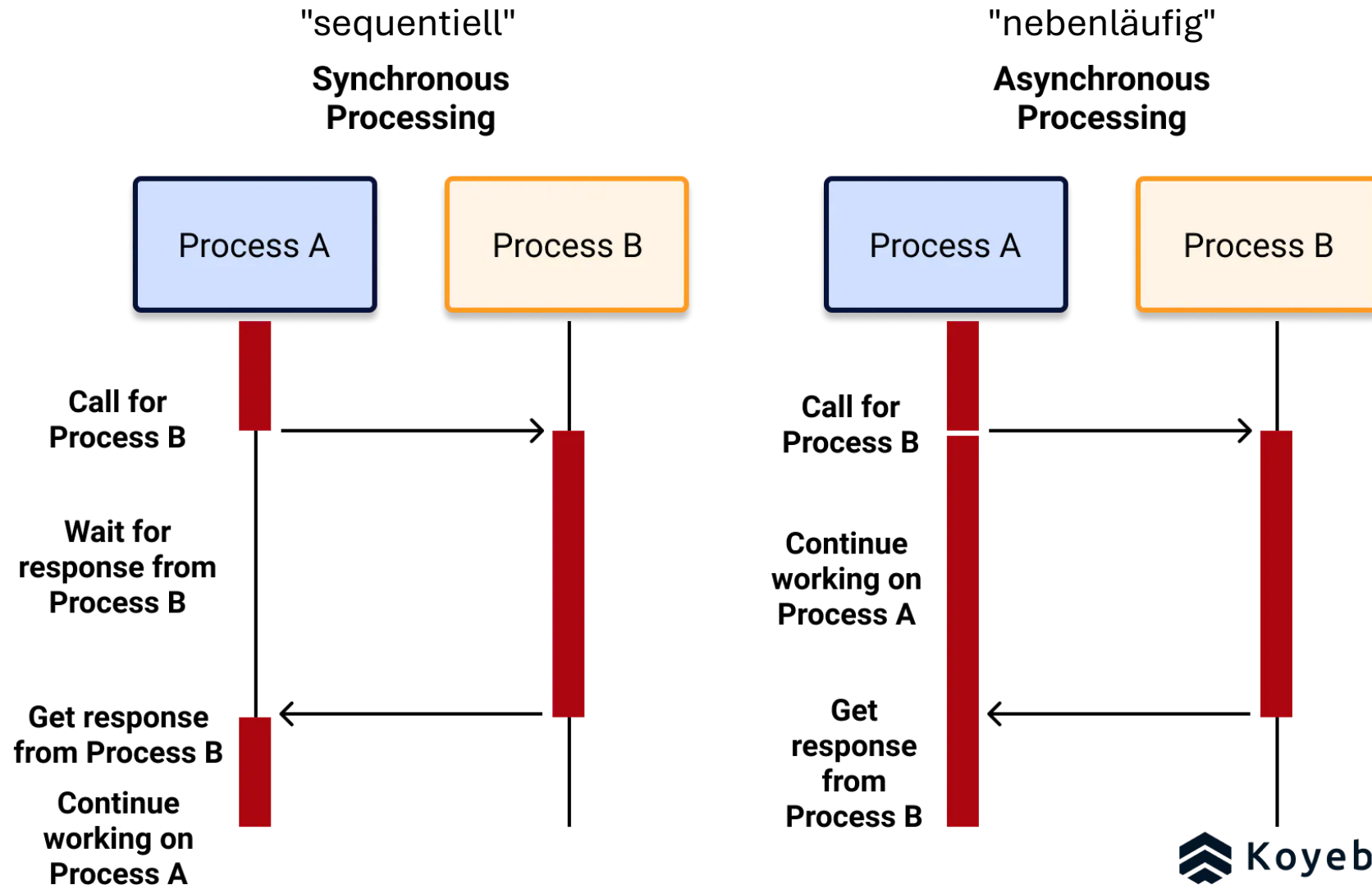
Future<V> ist ein **Interface** von Java und repräsentiert ein **zukünftiges Ergebnis** einer **asynchronen Berechnung**, d. h. das Java-Programm läuft weiter, während das Future ausgeführt wird.

Ein Future bietet Methoden, um den Status der Berechnung zu prüfen (isDone), die Berechnung abzubrechen (cancel) oder das Ergebnis abzurufen (**get**).

Die **get-Methode blockiert** den **aktuellen Thread** so lange, **bis das Ergebnis zur Verfügung steht** – ähnlich wie join() aus der Klasse Thread, nur das join() bei einem Thread kein Ergebnis zurückgibt.

→ Ein Future ist wie ein Thread, mit dem Unterschied, dass die blockierende zusätzliche get()-Methode ein Ergebnis zurückliefert.

Synchron vs Asynchron



Future<V>: Beispiel

Ein Future<V> wird **asynchron** ausgeführt. Das Ergebnis eines Futures erhält man über die get()-Methode. In diesem Beispiel würde der main-Thread so lange blockieren (warten), bis das Ergebnis von future.get() verfügbar ist.

```
void main() {  
    try (ExecutorService executor = Executors.newFixedThreadPool(2)) {  
        Future<String> future = executor.submit(() -> {  
            Thread.sleep(Duration.ofMillis(100));  
            return "A long calculation...";  
        });  
        // Do some other stuff in the meantime...  
        IO.println(future.get()); // blockiert, bis das Ergebnis verfügbar ist.  
    } catch (Exception e) {  
        throw new RuntimeException(e);  
    }  
}
```



```
@FunctionalInterface  
public interface Callable<V> {  
  
    V call() throws Exception;  
}
```

Ausgabe:

A long calculation...

Was ist ein ExecutorService? Verwaltet einen Pool von Plattform-Threads, um Aufgaben (Tasks) an diese zu verteilen, wobei die Threads zur Wiederverwendung vorgesehen sind (zur effizienteren Ausführung). ExecutorService sollte immer mit try-with-resources automatisch geschlossen werden. [#AutoClosable](#)

CompletableFuture<T>



CompletableFuture<T> (auch **Promise** oder **await/async**) ist (neben FutureTask) eine Java-Implementation von **Future<V>**.

Ein CompletableFuture wird ebenfalls **asynchron** ausgeführt, d. h., das Java-Programm läuft weiter, während das CompletableFuture ausgeführt wird.

Die **get()**-Methode **blockiert** den aktuellen Thread, bis das Ergebnis verfügbar ist. Sie wirft eine **Checked Exception**.

CompletableFuture<T> akzeptiert ein **Supplier<T>** und bietet neben get() zusätzlich die Methode **join()**, die wie get() auch **blockiert und ein Ergebnis zurückgibt**, aber nur **Unchecked Exceptions** wirft. 🤪

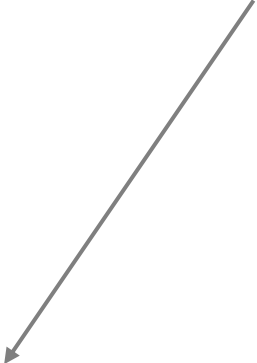
Darüber hinaus bietet CompletableFuture **Method-Chaining** (aka Fluent-API), um mehrere Aufgaben miteinander **verketteten** oder **kombinieren** zu können (z. B. supplyAsync, thenApply, exceptionally, thenAccept, thenRun, join).

CompletableFuture<T>: Beispiel

```
@FunctionalInterface
public interface Supplier<T> {

    T get();

}
```



```
void main() {
    CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> 42);
    // Awesome code, der ausgeführt wird, während das Future asynchron berechnet wird...
    IO.println(future.join()); // join() blockiert, bis future fertig ist und gibt dann das 42 als Rückgabewert zurück.
}
```

Hinweis: Im Gegensatz zu Future<V> verwendet CompletableFuture<T> das Interface **Supplier<T>** statt Callable<V>. Beide Interfaces sind fast gleich, Supplier<T> wirft allerdings keine Checked Exception.

Future<V> vs CompletableFuture<T>

Erzeugung:

```
try(ExecutorService executor = Executors.newFixedThreadPool(4)) {  
    Future<V> future = executor.submit(Callable<V> callable);  
    // ...  
}
```

```
CompletableFuture<T> completableFuture = CompletableFuture.supplyAsync(Supplier<T> supplier);
```

Verwendung:

```
V result = future.get(); // blocks current thread, throws Checked Exception
```

```
T result = completableFuture.join(); // blocks current thread, throws Unchecked Exception
```

```
T result = completableFuture.get(); // blocks current thread, throws Checked Exception
```

↑
CompletableFuture<T> implementiert
(ist auch) ein Future<T>.

CompletableFuture<T>: Beispiel Chaining 1/2

```
void main() {  
    CompletableFuture<String> future = CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            IO.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            IO.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            IO.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde  
            System.err.println(ex.getMessage());  
            return null;  
        })  
  
    IO.println("Join...");  
    var result = future.join(); // Blockiert, bis Ergebnis "Hello World!" verfügbar ist. Im Gegensatz zu get(), wirft join() nur Unchecked Exceptions.  
    IO.println(result);  
    IO.println("End.");  
}
```

Ausgabe:

```
Task 1  
Join...  
Task 2  
Task 3  
Hello World!  
End.
```

CompletableFuture<T>: Beispiel Chaining 2/2

```
void main() {  
    CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            IO.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            IO.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            IO.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde.  
            System.err.println(ex.getMessage());  
            return null;  
        })  
        .thenAccept(result -> IO.println(result)) // Verarbeitet das Endergebnis.  
        .thenRun(() -> IO.println("End."))  
        .join(); // Blockiert den main-Thread, ansonsten kann es sein, dass das CompletableFuture abgebrochen wird (shutdown).  
}
```

Ausgabe:

```
Task 1  
Task 2  
Task 3  
Hello World!  
End.
```

In dieser Variante wird alles in der CompletableFuture-Chain durchgeführt.



Daemon vs Non-Daemon Threads

Das `join()` am Ende des vorherigen Beispiels blockiert den `main-Thread`, bis das `CompletableFuture` abgeschlossen ist.

Ohne das `join()` kann es passieren, dass der `main-Thread` zuerst endet und dadurch das komplette Programm sofort beendet wird, obwohl das `CompletableFuture` noch läuft!

Das liegt daran, dass ein `CompletableFuture` als **daemon** Thread gestartet wird. BTW: Mit der Methode `setDaemon(boolean)` in der Klasse `Thread` lässt sich vorm Start einstellen, ob ein Thread ein daemon sein soll.

Sobald der `main-Thread` endet, gibt es keine **non-daemon** Threads mehr, was zu einem automatischen Shutdown der JVM führt. Alle laufenden **daemon**-Threads werden beim Shutdown der JVM einfach abgebrochen.



Exkurs: Promise (JavaScript)

```
function fetchPokemonData() {  
  fetch("https://pokeapi.co/api/v2/pokemon/pikachu") // Nicht-blockierende Funktion. Gibt ein Promise<Response> zurück mit dem zukünftigen Ergebnis.  
    .then(response => { // "zukünftiges Ergebnis"  
      if (!response.ok)  
        throw new Error('Fehler: ' + response.statusText);  
      return response.json();  
    })  
    .then(pokemon => {  
      const abilityUrl = pokemon.abilities[0].ability.url;  
      return fetch(abilityUrl);  
    })  
    .then(response => {  
      if (!response.ok)  
        throw new Error('Fehler: ' + response.statusText);  
      return response.json();  
    })  
    .then(ability => {  
      console.log("Details zur Fähigkeit:", ability);  
    })  
    .catch(error => {  
      console.error("Fehler beim Abrufen der Daten:", error);  
    })  
    .finally(() => {  
      console.log("End");  
    });  
}
```

In JavaScript gibt es auch **CompletableFuture<T>**, dort heißt das **Promise<T>**.

Die Funktion **fetch** ruft hier den Inhalt einer URL ab und gibt den Inhalt als Promise zurück.



ACHTUNG:

Das Beispiel ist kein Java sondern ein JavaScript!



Exkurs: async/await (JavaScript)

```
async function fetchPokemonData() { // Nicht-blockierende Funktion.
  try {
    const pokemonResponse = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");
    if (!pokemonResponse.ok)
      throw new Error('Fehler: ' + pokemonResponse.statusText);
    const pokemon = await pokemonResponse.json();

    const abilityUrl = pokemon.abilities[0].ability.url;
    const abilityResponse = await fetch(abilityUrl);
    if (!abilityResponse.ok)
      throw new Error('Fehler: ' + abilityResponse.statusText);
    const ability = await abilityResponse.json();

    console.log("Details zur Fähigkeit:", ability);
  } catch (error) {
    console.error("Fehler beim Abrufen der Daten:", error);
  } finally {
    console.log("End");
  }
}
```

async/await ist eine Alternative Schreibweise für ein Promise in JavaScript (die Schlüsselwörter gibt es z. B. auch in Python, C# und Rust).



ACHTUNG:

Das Beispiel ist kein Java sondern ein JavaScript!

HttpClient

Ein HttpClient ist eine Klasse in Java, die **CompletableFuture** für **asynchronone** Webanfragen verwendet.

Mit HttpClient kann eine Anfrage an eine URI gesendet werden (**request**). Die Antwort der Abfrage wird als CompletableFuture zurückgegeben (**response**).

HttpClient: Asynchrones Beispiel

Die Abfrage wird **asynchron** ausgeführt. Mit **join** wird auf das **fertige Ergebnis** aus dem `CompletableFuture<T>` **gewartet**.

```
class HttpClientExample {  
  
    void main() {  
        try (HttpClient client = HttpClient.newBuilder().build()) {  
            client.sendAsync(createRequest("https://boardgamegeek.com/browse/boardgame/toplist.html"),  
                            HttpResponse.BodyHandlers.ofString()) // CompletableFuture<HttpResponse<String>>  
                .thenApply(HttpResponse::body) // CompletableFuture<String>  
                .thenAccept(IO::println) // CompletableFuture<Void>  
                .join();  
        }  
    }  
  
    private static HttpRequest createRequest(String uri) {  
        return HttpRequest.newBuilder().uri(URI.create(uri)).build();  
    }  
}
```



FYI: `HttpClient.sendAync` ist quasi das Äquivalent in Java zu `fetch` in JavaScript.

Recap

Thread: Java-Klasse, mit der Code nebenläufig ausgeführt werden kann (verwendet Runnable-Interface).

synchronized: Java-Schlüsselwort um thread-safe zu programmieren. Für feingranulare Optimierung.

Collections.synchronized*: Macht eine beliebige Collection-Klasse thread-safe.

Concurrent: Package mit vielen nützlichen thread-safe Klassen.

ExecutorService: Verteilt Aufgaben (Tasks) an einen Thread-Pool. Threads werden dabei wiederverwendet.

Runnable: Functional Interface, ohne Rückgabewert, das als Thread ausgeführt werden kann.

Callable: Functional Interface ähnlich wie Runnable, das aber ein Ergebnis zurückgibt (wie Supplier<T>). Kann von ExecutorService als Thread ausgeführt werden und gibt ein Future<V> zurück.

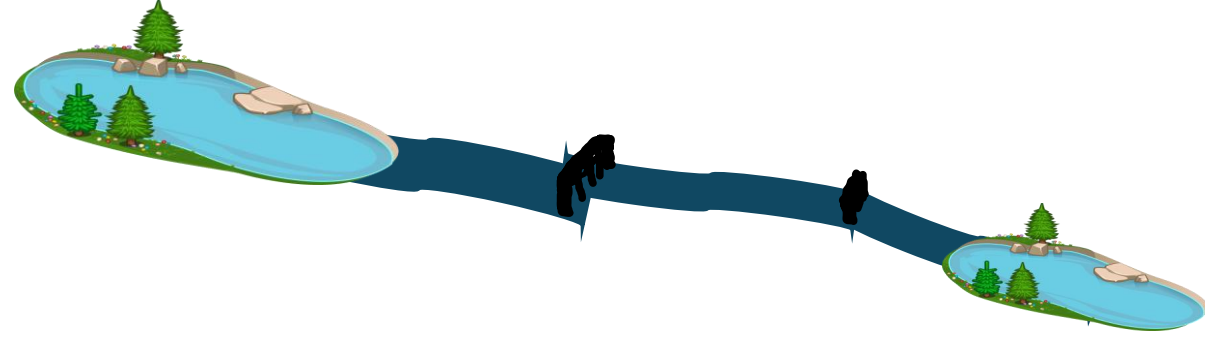
Supplier wirft aber keine Exception.

Future<V> (aka Promise bzw. await/async): Interface. Berechnet Ergebnis asynchron. Methode get() blockiert und liefert Ergebnis, wirft aber Checked Exception. Beispiel: Future<V> executor.submit(Callable<V> task);

CompletableFuture<T>: Implementiert Future<V> mit Fluent-API. Verwendet statt Callable<V> das Supplier<T>-Interface. Methoden get() und join() liefern das Ergebnis (blockierend), join() wirft aber nur eine Unchecked Exception.

HttpClient: Internet-Ressourcen anhand einer URI abrufen. Jede Anfrage (request) gibt ein CompletableFuture<T> (response) mit dem Ergebnis zurück.

Back to Streams



Standardmäßig wird jede Stream-Operation in Java sequenziell verarbeitet:

```
List<Integer> oneToNine = IntStream.range(1, 10).boxed().toList();  
oneToNine.stream().forEach(number -> IO.print(number + " "));
```

Was wird ausgegeben? 🤔

Back to Streams: Ausgabe



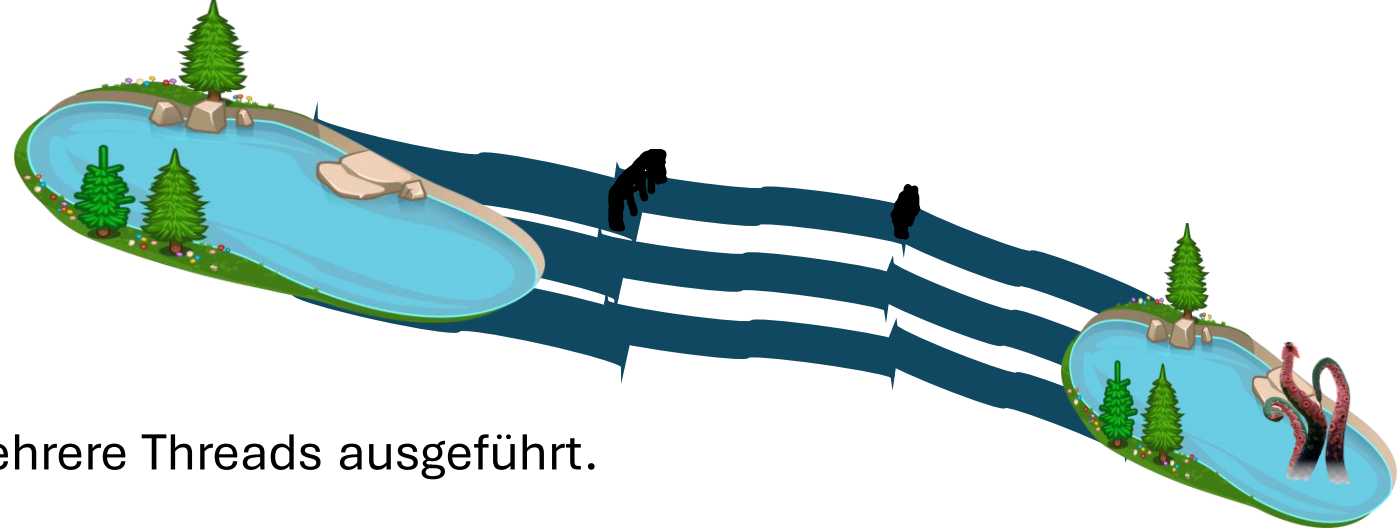
Standardmäßig wird jede Stream-Operation in Java sequenziell verarbeitet:

```
List<Integer> oneToNine = IntStream.range(1, 10).boxed().toList();  
oneToNine.stream().forEach(number -> IO.print(number + " "));
```

Ausgabe:

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Parallel-Stream



Parallel-Streams werden intern durch mehrere Threads ausgeführt.

```
List<Integer> oneToNine = IntStream.range(1, 10).boxed().toList();  
oneToNine.parallelStream().forEach(number -> IO.print(number + " "));
```

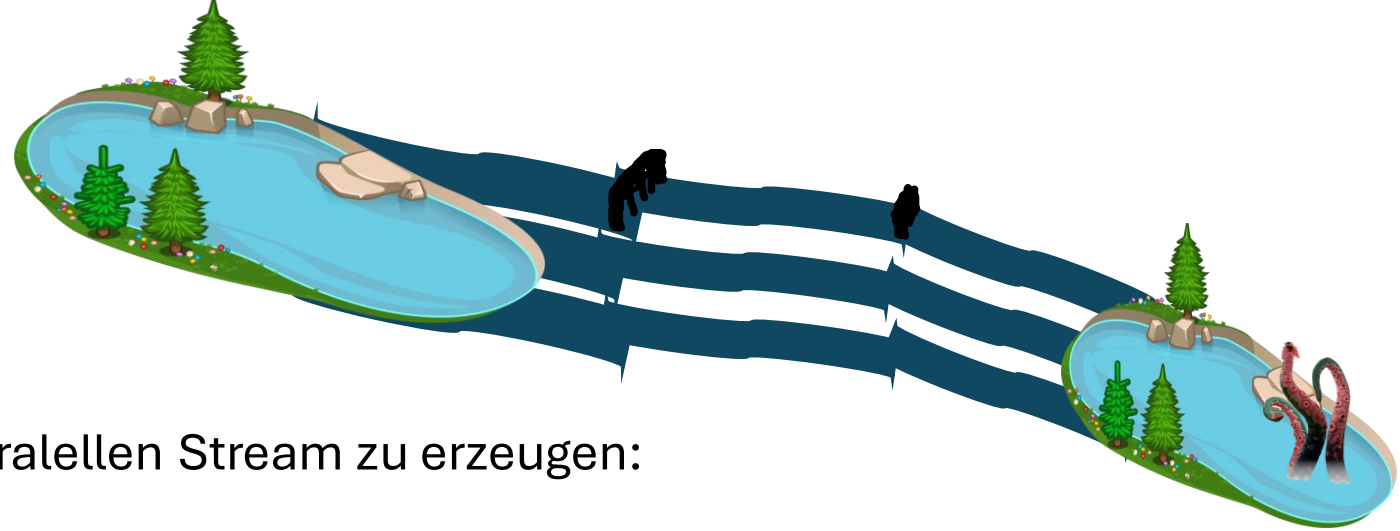
Mögliche Ausgabe:

6	3	4	2	1	7	9	8	5
---	---	---	---	---	---	---	---	---

Es dürfen **keine Seiteneffekte** innerhalb eines Parallel-Streams auftreten (siehe **Pure Lambda**). Wenn alle Lambdas pure sind, dann lässt sich fast jeder Stream durch einen `parallelStream` ersetzen.

Ausnahme: Reihenfolgeabhängige Operationen wie z. B. `limit`, `skip`, `reduce` müssen sorgfältig geprüft werden, ob sie das gewünschte Ergebnis liefern.

Parallel-Stream



Es gibt zwei Schreibweisen, um einen parallelen Stream zu erzeugen:

```
List<Integer> oneToNine = IntStream.range(1, 10).boxed().toList();  
oneToNine.parallelStream().forEach(number -> IO.print(number + " "));  
oneToNine.stream().parallel().forEach(number -> IO.print(number + " "));
```

Mögliche Ausgabe:

6	3	4	2	1	7	9	8	5
---	---	---	---	---	---	---	---	---

Wiederholung: Pure Lambda



In Java sind Seiteneffekte mit Lambdas (not pure) möglich, dies ist allerdings eine Quelle für Bugs, insbesondere bei parallelen Streams!

```
void main() {  
    var names = List.of("Elona", "Mark", "Bill", "Jeff", "Sara", "Lana");  
  
    for (int i = 0; i < 10; i++) {  
        var result = new ArrayList<String>();  
        names.stream().parallel() // wandle sequenziellen Stream in parallelen Stream um.  
            .forEach(name -> result.add(name)); // result wird verändert (Seiteneffekt).  
  
        IO.println("Size of result: " + result.size() + "/" + names.size());  
    }  
}
```

Mögliche Ausgaben:

Size of result: 5/6
Size of result: 4/6
Size of result: 5/6
Size of result: 5/6
Size of result: 5/6
Size of result: 5/6
Size of result: 1/6
Size of result: 5/6
Size of result: 4/6
Size of result: 5/6

In diesem Beispiel sollte result immer 6 als size() zurückgeben, durch den Seiteneffekt und die parallele Ausführung werden jedoch manche Namen nicht hinzugefügt.

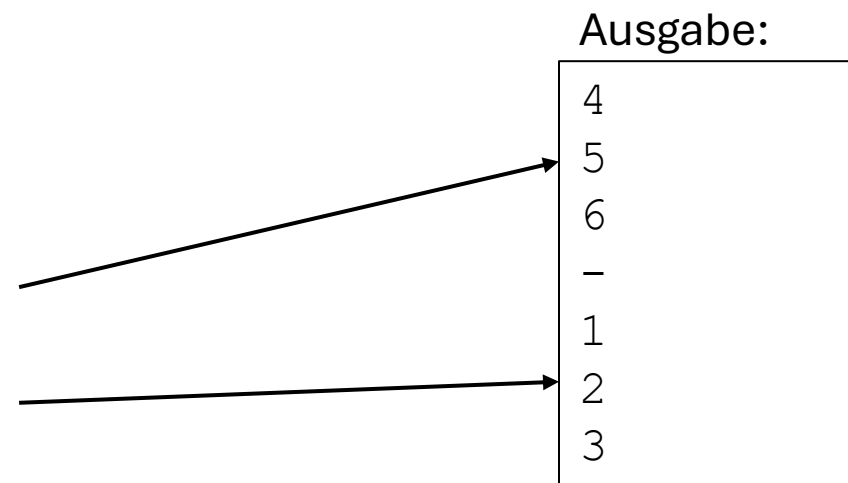


Exkurs: Spliterator

Parallel-Streams verwenden intern das Interface [Spliterator](#)<T>, um **Daten auf mehrere Threads zu verteilen**. Jeder Thread führt anschließend alle Stream-Anweisungen mit seiner zugewiesenen Teilmenge nebenläufig aus.

Die JVM versucht dabei die Daten so aufzuteilen, dass die vorhandene CPU-Kerne möglichst optimal ausgenutzt werden. Die konkrete Implementierung des Spliterators ist abhängig von der jeweiligen Collection-Klasse.

```
void main() {  
    List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);  
  
    Spliterator<Integer> part1 = numbers.spliterator();  
    Spliterator<Integer> part2 = part1.trySplit();  
  
    part1.forEachRemaining(number -> IO.println(number));  
    IO.println("-");  
    part2.forEachRemaining(number -> IO.println(number));  
}
```



To parallel or not to parallel?



Die parallele Verarbeitung kann Vorteilhaft sein, um mehrere Rechenkerne voll auszunutzen.

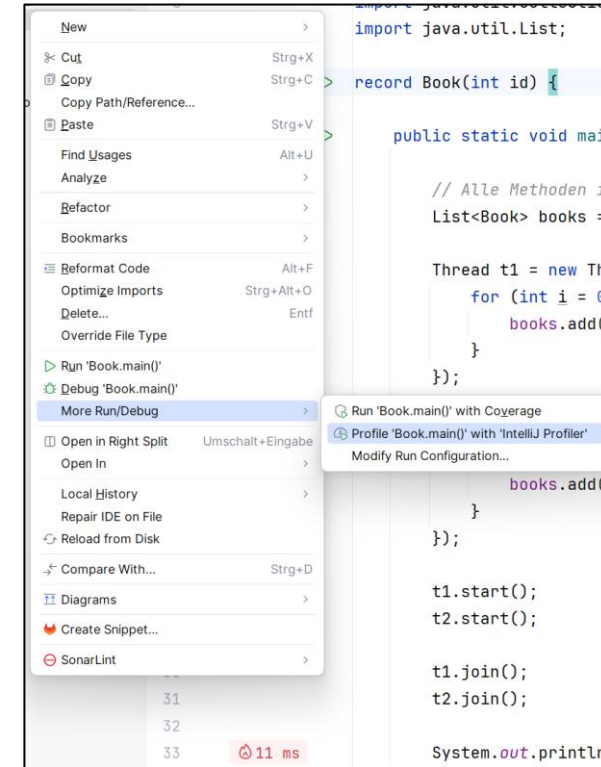
Der **Overhead** ist allerdings zu berücksichtigen:

- In the NQ model, **N stands for the number of source data elements**, while **Q represents the amount of computation** performed per data element.
 - The larger the product of $N*Q$, the more likely we are to get a performance boost from parallelization.
 - For problems with a trivially small Q , such as summing up numbers, the rule of thumb is that N should be greater than 10,000. **As the number of computations increases, the data size required to get a performance boost from parallelism decreases.**
- Parallelism can bring performance benefits in certain use cases. But parallel streams cannot be considered as a magical performance booster. So, **sequential streams should still be used as default during development.**

To parallel in der Praxis

In der Praxis misst man die Ausführungszeit für das sequenzielle und das parallele Programm und vergleicht diese anschließend:

```
Instant start = Instant.now();  
// AWESOME (sequential or parallel) CODE  
Instant finish = Instant.now();  
long elapsedTimeInMs = Duration.between(start, finish).toMillis();  
IO.println(elapsedTimeInMs + " ms");
```



BTW: Langsame Stellen bzw. "**Bottlenecks**" lassen sich im Code mit [Profiler-Tools](#) identifizieren (z. B. in IntelliJ Ultimate Edition vorhanden).

Merke: Häufig reicht es aus, nur die Bottlenecks zu optimieren (z. B., durch parallelisieren).

#[MostBangForTheBuck](#)